

固废管理系统（Solid Waste Management System）设计方案

无害化处理厂电子台账系统 — 覆盖固废从接收→处置→贮存转运→检测的全流程数字化管理

1. 系统概述

1.1 业务背景

本系统服务于无害化处理厂，将传统的纸质台账电子化，实现固废全生命周期的数字化追踪管理。

1.2 核心业务流程

产废单位出废 → 运输到厂 → 过磅接收(接收台账) → 无害化处置(处置台账)

↓

└─ 入库贮存(贮存台账) ←

↓

└─ 库位转运(转运台账)

↓

└─ 各环节检测(检测台账)

1.3 4个子系统

子系统	录入角色	核心数据
接收台账	库管 + 司机	批次号、产废单位、毛重/皮重/净重、车牌号、接收日期
处置台账	总控制室操作员	处置工艺、入炉重量、产出重量、温度、设备
贮存转运台账	库管 + 司机	入库/出库/盘点、库存、转运单
检测台账	检测员	检测项目、方法、结果、合格判定、复核

1.4 角色定义

角色	编码	职责范围
管理员	ADMIN	全部权限（老板/系统管理员）
库管	KEEPER	接收台账、贮存台账出入库盘点、转运台账
司机	DRIVER	仅查看自己的运输记录
总控制室操作员	OPERATOR	处置台账录入
检测员	TESTER	检测台账录入与复核

2. 技术架构

2.1 技术栈 (JDK8 兼容)

后端: JDK 1.8 + Spring Boot 2.7.18 + Spring Security 5.7 + MyBatis-Plus 3.5 + MySQL 8.0
前端: Vue 2.7 + Element UI 2.15 + Axios + Vue Router 3 + Vuex 3
工具: Hutool 5.8 + EasyExcel 3.3 + Knife4j 3.0
构建: Maven 3.8 + Vue CLI 5

2.2 项目结构

```
SolidwasteManager/  
├── swm-backend/           # Maven 父工程  
│   ├── swm-common/       # 公共模块  
│   ├── swm-security/     # 安全认证模块  
│   ├── swm-system/       # 系统管理模块  
│   ├── swm-business/     # 业务台账模块  
│   ├── swm-dict/         # 字典模块  
│   ├── swm-api/          # 启动入口  
│   └── doc/sql/           # SQL脚本  
├── swm-frontend/         # Vue 2 前端工程  
└── doc/                  # 项目文档
```

2.3 架构分层

Vue前端(SPA) → Nginx(反向代理+静态资源) → SpringBoot REST API
→ Controller → Service → Mapper(MyBatis-Plus) → MySQL 8.0

安全: Spring Security + JWT (无状态认证)

API: RESTful, 统一返回 { code, message, data, timestamp }

权限: RBAC (用户→角色→菜单/按钮)

3. 数据库设计

3.1 系统表 (5张)

sys_user — 用户表

- user_id, username(UNIQUE), password(BCrypt), real_name, phone, email
- status(0禁用/1启用), is_deleted(逻辑删除), last_login_time/ip
- 初始用户: admin / admin123

sys_role — 角色表

- role_id, role_code(ADMIN/KEEPER/DRIVER/OPERATOR/TESTER), role_name, description

sys_menu — 菜单权限表

- menu_id, parent_id(树形), menu_name, menu_type(M目录/C菜单/B按钮)

- permission(权限标识如 receiving:add), path(路由), component(组件路径), icon

sys_user_role — 用户角色关联 (user_id, role_id, UNIQUE)

sys_role_menu — 角色菜单关联 (role_id, menu_id, UNIQUE)

3.2 字典表 (5张)

- **swm_waste_producer**: 产废单位 (producer_code, producer_name, license_no, license_suffix用于批次号)
- **swm_workshop**: 车间 (workshop_code, workshop_abbr用于批次号, workshop_name)
- **swm_mine_source**: 矿源 (mine_code用于批次号, mine_name)
- **swm_waste_category**: 危废类别 (category_code, category_name)
- **swm_treatment_process**: 处置工艺 (process_code, process_name)

3.3 业务台账表 (5张 + 1张日志表)

swm_receiving — 接收台账

- batch_no(UNIQUE, 系统自动生成), producer_id, workshop_id, mine_source_id, waste_category_id
- plate_number(车牌号), driver_id(FK→sys_user)
- gross_weight, tare_weight, net_weight(净重=毛重-皮重, >0)
- receive_date, receive_time, receive_user_id(FK→sys_user)
- status: 1已接收→2处置中→3已处置→4已完结

swm_treatment — 处置台账

- batch_no, process_id, treatment_date, start_time, end_time
- input_weight, output_weight, treatment_loss(损耗=投入-产出)
- equipment_name/code, temperature, additive_name/weight
- operator_id(FK→sys_user), status(1处置中/2处置完成)

swm_storage — 贮存台账

- batch_no, storage_location(库位), storage_type(1入库/2出库/3盘点)
- change_weight(正入负出), current_weight(变动后库存), operation_date
- related_transfer_id(关联转运单), operator_id

swm_transfer — 转运台账

- transfer_no(UNIQUE, ZY+日期+流水), batch_no
- from_location, to_location, transfer_weight, transfer_date
- vehicle, driver_id, operator_id

swm_testing — 检测台账

- testing_no(UNIQUE, JC+日期+流水), batch_no
- testing_date, sample_name/location, testing_item/method
- standard_value, testing_result, result_unit, is_qualified(0否/1是)

- tester_id, reviewer_id(复核人≠检测人), review_time, status(1待复核/2已复核)

swm_operation_log — 操作日志

- user_id, username, module, operation(ADD/UPDATE/DELETE/EXPORT/LOGIN)
- description, request_method/url/params, ip_address, cost_time

3.4 批次号生成规则

批次号 = license_suffix(许可证后5位) + "-" + workshop_abbr(车间缩写) + "-" + mine_code(矿源编号3位) + "-" + 日期(yyyyMMdd)
示例: 12345-AB-001-20260520

重复处理: 同组合自动追加 "-01", "-02"
示例: 12345-AB-001-20260520-01

4. API 接口设计

4.1 统一规范

- URL: /api/v1/{module}[//{id}][/{action}]
- 分页: ?page=1&size=20&sortField=createTime&sortOrder=desc
- 认证: Authorization: Bearer <JWT Token>
- 响应: { code: 200, message: "成功", data: {...}, timestamp: 1716211200000 }

4.2 接口列表

模块	方法	URL	权限	说明
认证	POST	/api/v1/auth/login	无	登录
认证	GET	/api/v1/auth/userinfo	登录用户	获取用户信息+菜单
认证	POST	/api/v1/auth/logout	登录用户	登出
接收台账	GET	/api/v1/receiving	receiving:list	分页查询
接收台账	GET	/api/v1/receiving/{id}	receiving:list	详情
接收台账	POST	/api/v1/receiving	receiving:add	新增
接收台账	PUT	/api/v1/receiving/{id}	receiving:edit	编辑
接收台账	DELETE	/api/v1/receiving/{id}	receiving:delete	删除
接收台账	GET	/api/v1/receiving/export	receiving:export	导出Excel
处置台账	GET/POST/PUT/DELETE	/api/v1/treatment[:id]	treatment:*	CRUD
处置台账	PUT	/api/v1/treatment/{id}/complete	treatment:edit	完成处置
贮存台账	GET	/api/v1/storage	storage:list	流水查询
贮存台账	GET	/api/v1/storage/inventory	storage:list	库存总览

模块	方法	URL	权限	说明
贮存台账	POST	/api/v1/storage/in	storage:in	入库
贮存台账	POST	/api/v1/storage/out	storage:out	出库
贮存台账	POST	/api/v1/storage/check	storage:check	盘点
转运台账	GET/POST/DELETE	/api/v1/transfer[:id]	transfer:*	CRUD
检测台账	GET/POST/PUT/DELETE	/api/v1/testing[:id]	testing:*	CRUD
检测台账	PUT	/api/v1/testing/{id}/review	testing:review	复核
检测台账	GET	/api/v1/testing/statistics	testing:list	合格率统计
系统	GET/POST/PUT/DELETE	/api/v1/system/users[:id]	system:user	用户管理
系统	GET/POST/PUT/DELETE	/api/v1/system/roles[:id]	system:role	角色管理
系统	GET/POST/PUT/DELETE	/api/v1/system/menus[:id]	system:menu	菜单管理
系统	GET	/api/v1/system/menus/tree	(登录用户)	菜单树
字典	GET/POST/PUT/DELETE	/api/v1/dict/{type}[:id]	dict:*	CRUD
字典	GET	/api/v1/dict/{type}/list	(登录用户)	下拉列表
仪表盘	GET	/api/v1/dashboard/statistics	dashboard:view	统计数据

5. 权限设计（RBAC）

5.1 权限矩阵

功能	ADMIN	KEEPER	DRIVER	OPERATOR	TESTER
仪表盘	R	R	R	R	R
接收台账 CRUD	全部	CU	R(自己)	R	R
处置台账 CRUD	全部	-	-	CU	R
贮存台账 CRUD	全部	CU	-	R	R
转运台账 CRUD	全部	CU	R(自己)	-	R
检测台账 CRUD	全部	-	-	-	CU
检测复核	可	-	-	-	可
系统管理	全部	-	-	-	-
字典管理	全部	-	-	-	-
操作日志	R	-	-	-	-

5.2 三层权限控制

- 1. 路由层: 前端根据后端返回 menus 动态 addRoutes
- 2. 视图层: `v-permission="'receiving:add'"` 指令控制按钮
- 3. API层: `@PreAuthorize("hasAuthority('xxx')")` 接口守卫

5.3 JWT 认证

登录 → BCrypt校验 → 生成JWT(HS256,24h) → 前端存localStorage

→ 请求Header: Authorization: Bearer <token>

→ JwtAuthFilter校验 → 注入SecurityContext

→ 登出时Token加入Redis黑名单

6. 前端设计

6.1 页面结构

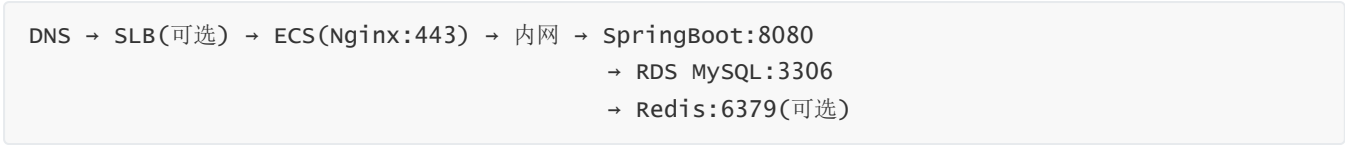
/login	登录页
/dashboard	首页仪表盘（ECharts数据卡片+趋势图）
/receiving/list	接收台账列表+新增/编辑弹窗
/treatment/list	处置台账列表+完成处置
/storage/list	贮存台账+库存总览
/transfer/list	转运台账
/testing/list	检测台账+复核
/system/user	用户管理
/system/role	角色管理
/system/menu	菜单管理
/system/log	操作日志
/dict/producer	产废单位字典
/dict/workshop	车间字典
/dict/mine	矿源字典
/dict/category	危废类别字典
/dict/process	处置工艺字典
/profile	个人中心

6.2 关键机制

- **Axios拦截器:** 请求注入Token, 响应统一错误/Token过期处理
- **路由守卫:** beforeEach → 验证Token → 获取用户信息 → 动态生成路由
- **按钮权限:** v-permission指令, 无权限DOM直接移除
- **字典缓存:** Vuex store 中缓存字典下拉数据, DictSelect组件统一消费

7. 部署方案

7.1 云部署拓扑



7.2 简化版（初期）

1台ECS(CentOS 7.9, 2C4G)

- └─ Nginx: 反向代理 + 静态资源(Vue dist/) + HTTPS
- └─ Java -jar swm-api.jar (systemd管理)
- └─ MySQL 可用RDS(推荐)或同机安装

7.3 Android适配

后端接口完全不变，Android通过 Retrofit+OkHttp 消费同一套 REST API，离线场景 Room 本地缓存待提交数据。

8. 实施计划

阶段	内容	预估
1	项目骨架 + 登录认证	3天
2	接收台账 + 字典管理	4天
3	处置台账	3天
4	贮存+转运台账	4天
5	检测台账 + 仪表盘	4天
6	系统管理 + 联调	3天
7	部署上线	2天
合计		~23工作日